# LAPORAN RESMI PRAKTIKUM

## DevSecOps — Bab 5

### Database Service di Docker: PostgreSQL

Nama : Fawwaz Mufid Wardaya
Kelas : D4 Teknik Informatika B
NRP : 3126640050

**PROGRAM STUDI TEKNIK INFORMATIKA**
**DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER**
**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**
**2026**

# Pendahuluan

## Latar Belakang

Database container berbeda dari layanan stateless: data adalah aset utama sehingga volume, backup, restore, permission, dan lifecycle container harus dirancang dengan hati-hati. Bab 5 menjalankan PostgreSQL pada Docker dengan konfigurasi berbasis environment variable dan volume persisten, menginisialisasi skema melalui docker-entrypoint-initdb.d, mengakses database dengan psql dan pgAdmin4, menerapkan healthcheck, serta menguji backup dan restore secara nyata. Penekanan praktikum adalah membuktikan bahwa data bertahan melampaui umur container dan bahwa backup benar-benar dapat dipulihkan, bukan sekadar diasumsikan berhasil.

## Rumusan Masalah

Mengapa init script tidak dijalankan ulang saat volume lama masih ada?

Apa risiko menaruh password database pada docker-compose.yml?

Bagaimana cara membuktikan backup dapat dipulihkan?

Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?

Apa dampak docker compose down -v terhadap database?

# Pembahasan

## Dasar Teori

PostgreSQL di container tetap sistem database relasional penuh; container hanya mengubah cara proses dan dependensinya dikemas. Data tidak boleh dianggap melekat pada umur container, sehingga volume persisten diperlukan. Persistensi memiliki empat dimensi: durability transaksi, keberlangsungan media, kemampuan backup, dan kemampuan restore. Volume yang persisten tetap bukan backup, sehingga uji restore ke instance atau database terpisah wajib dilakukan. Konfigurasi image PostgreSQL dikendalikan environment variable berikut.

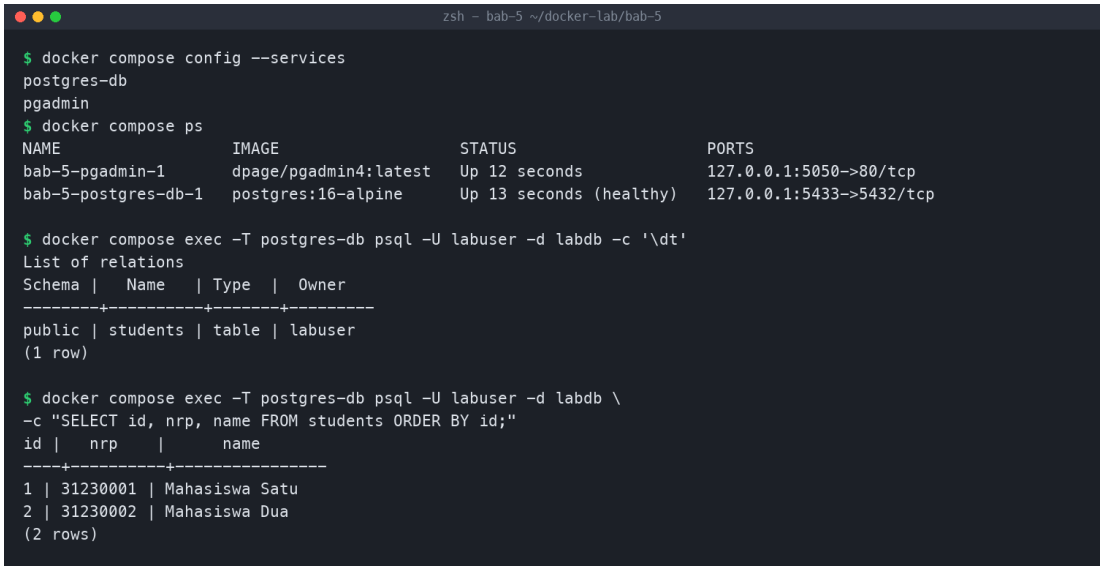
| Variabel | Fungsi |
|---|---|
| POSTGRES_DB | Nama database awal yang dibuat saat inisialisasi |
| POSTGRES_USER | User awal/superuser container |
| POSTGRES_PASSWORD | Password awal; wajib pada lab sederhana |
| PGDATA | Direktori data PostgreSQL di dalam container |
| POSTGRES_INITDB_ARGS | Argumen initdb, misalnya encoding atau locale |

Image resmi mengeksekusi file .sql dan .sh di /docker-entrypoint-initdb.d hanya ketika volume data masih kosong. Model ACID menjaga transaksi, dan constraint (primary key, unique, not null) menolak state tidak valid meskipun aplikasi memiliki bug. Dari sisi keamanan, prinsip least privilege mengharuskan akun aplikasi hanya memperoleh hak yang diperlukan; akun superuser tidak boleh dipakai aplikasi harian. Credential tidak boleh ditanam pada image atau repository, dan port database sebaiknya hanya terikat ke loopback atau network internal.

## Praktikum 1 — Stack PostgreSQL + pgAdmin, inisialisasi schema, dan konflik port

**Perintah:**

```
docker compose config --services
docker compose up -d
docker compose ps
docker compose exec -T postgres-db psql -U labuser -d labdb -c "\dt"
docker compose exec -T postgres-db psql -U labuser -d labdb \
  -c "SELECT id, nrp, name FROM students ORDER BY id;"
```



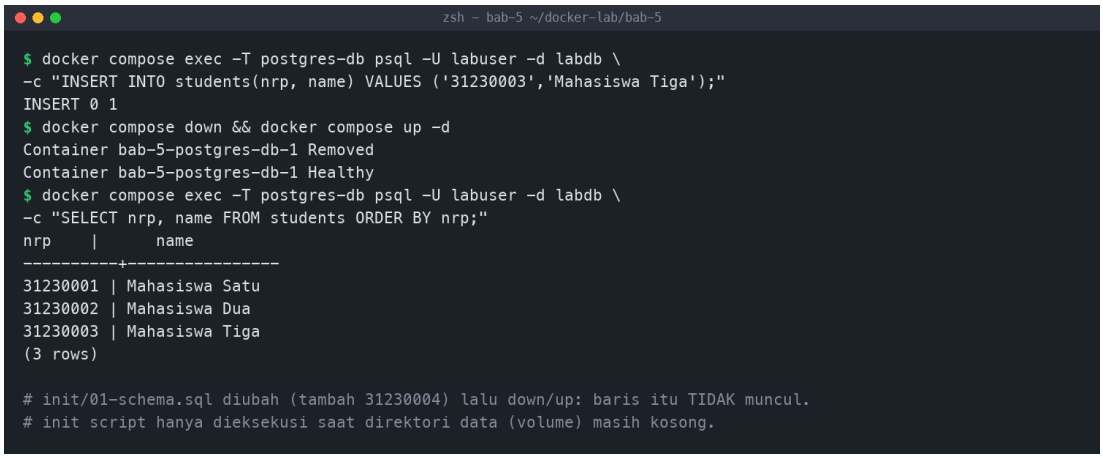

*Gambar 1. Stack berjalan; init script membuat tabel students dan dua baris data awal.*

**Analisis:** Compose menyalakan postgres-db dan pgadmin pada network data-net. Healthcheck pg_isready membuat postgres-db berstatus healthy, dan pgadmin menunggu kondisi tersebut sebelum start. Init script 01-schema.sql dieksekusi otomatis saat volume masih kosong sehingga tabel students beserta dua baris data awal tersedia. Kedua port dipublikasikan hanya pada loopback. Catatan kendala: up pertama gagal dengan "Bind for 0.0.0.0:5432 failed: port is already allocated" karena port 5432 host telah dipakai container lain; diagnosis melalui docker ps, dan diperbaiki dengan memindahkan host port ke 127.0.0.1:5433:5432 tanpa mengubah konektivitas internal.

## Praktikum 2 — Persistensi volume dan batasan init script

**Perintah:**

```
docker compose exec -T postgres-db psql -U labuser -d labdb \
  -c "INSERT INTO students(nrp,name) VALUES ('31230003','Mahasiswa Tiga');"
docker compose down
docker compose up -d
docker compose exec -T postgres-db psql -U labuser -d labdb \
  -c "SELECT nrp, name FROM students ORDER BY nrp;"
```




*Gambar 2. Data bertahan setelah container dibuat ulang; perubahan init script tidak berpengaruh.*

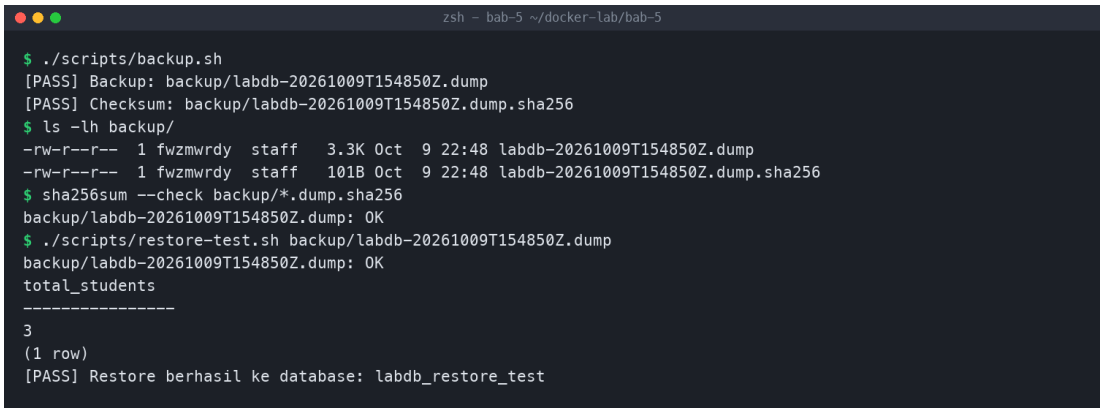
**Analisis:** Record 31230003 tetap ada setelah docker compose down lalu up, membuktikan named volume pg-data bertahan melampaui lifecycle container. Percobaan menambahkan record 31230004 ke init script lalu membuat ulang container tidak menampilkan record tersebut. Ini membuktikan init script hanya dieksekusi saat direktori data kosong: entrypoint mendeteksi klaster yang sudah

terinisialisasi dan melewati inisialisasi demi melindungi data. Untuk perubahan pada database yang sudah ada, digunakan migrasi SQL, bukan mengandalkan init script.

## Praktikum 3 — Backup (pg_dump) dan uji restore (pg_restore)

**Perintah:**

```
./scripts/backup.sh
ls -lh backup/
sha256sum --check backup/*.dump.sha256
latest_dump="$(find backup -maxdepth 1 -name '*.dump' | sort | tail -n 1)"
./scripts/restore-test.sh "$latest_dump"
```

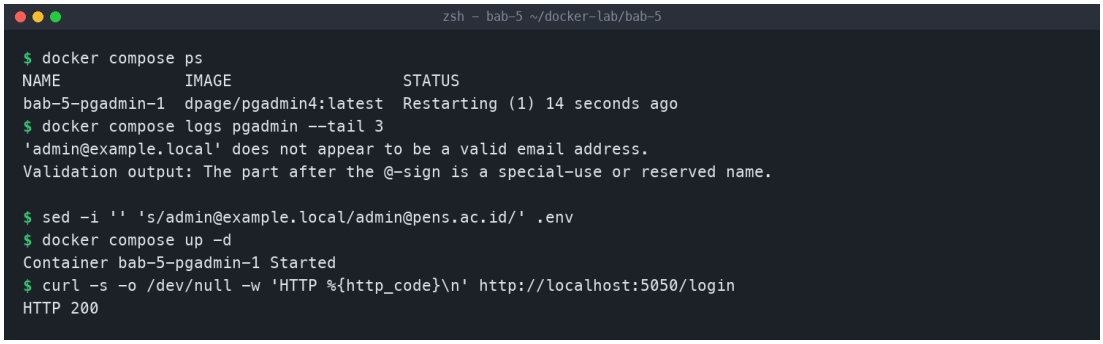



*Gambar 3. Backup custom-format beserta checksum, lalu restore terverifikasi ke database terpisah.*

**Analisis:** backup.sh menjalankan pg_dump --format custom melalui docker compose exec, memastikan tidak bergantung pada nama container dinamis. File dump berukuran lebih dari nol (3.3 KB) dan checksum SHA-256 valid. restore-test.sh memverifikasi checksum, membuat database labdb_restore_test, menjalankan pg_restore, lalu menghitung jumlah baris: total_students = 3, konsisten dengan sumber. Backup karena itu terbukti dapat dipulihkan, bukan hanya ada.

## Analisis Masalah: pgAdmin Terjebak Restart Loop

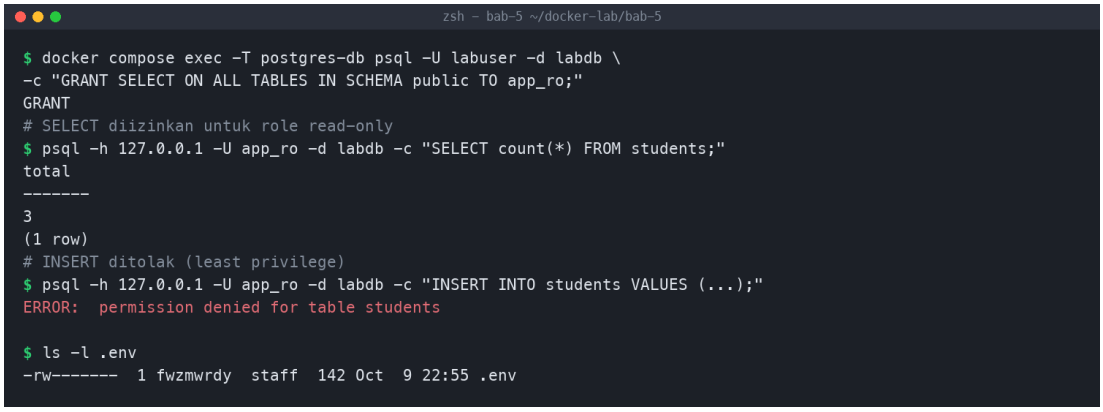
Sesaat setelah stack dinyalakan, container pgadmin tidak stabil dan terus restart (Restarting (1) berulang). Diagnosis dilakukan dengan membaca log: docker compose logs pgadmin menampilkan pesan "'admin@example.local' does not appear to be a valid email address" dengan keterangan bahwa bagian setelah tanda @ merupakan special-use atau reserved name. Nilai contoh PGADMIN_DEFAULT_EMAIL pada cheatsheet memakai domain .local (mDNS) yang ditolak validator email pgAdmin; entrypoint kemudian keluar dan container di-restart oleh kebijakan restart. Masalah ini adalah kesalahan konfigurasi, bukan kegagalan image. Perbaikan dilakukan dengan mengganti email menjadi domain valid (admin@pens.ac.id) dan membuat ulang container. Setelah itu pgAdmin berstatus Up dan halaman login mengembalikan HTTP 200. Pelajaran: nilai konfigurasi harus divalidasi, dan log container adalah sumber diagnosis pertama.




*Gambar 4. Restart loop pgAdmin terdiagnosis dari log lalu diperbaiki dengan email yang valid.*

### Keamanan: Least Privilege, Loopback, dan Proteksi Credential

Meskipun PostgreSQL berjalan sebagai superuser labuser, aplikasi seharusnya memakai role terbatas. Dibuat role app_ro dengan hak SELECT saja: role ini berhasil menjalankan SELECT (3 baris) tetapi ditolak saat INSERT dengan "ERROR: permission denied for table students". Port database dan pgAdmin dipublikasikan hanya pada loopback (127.0.0.1:5433 dan 127.0.0.1:5050), sehingga tidak terbuka ke jaringan. Credential disimpan pada .env dengan permission 600 dan dikecualikan dari version control melalui .gitignore, sehingga password serta dump tidak masuk repository.




*Gambar 5. Least privilege terbukti menolak tulis, dan .env terlindungi permission 600.*

### Troubleshooting

| Gejala | Penyebab yang mungkin | Tindakan korektif |
|---|---|---|
| port is already allocated | Port host 5432 dipakai container lain | Pindahkan ke 127.0.0.1:5433:5432 |
| pgAdmin restart loop | PGADMIN_DEFAULT_EMAIL domain reserved (.local) | Ganti ke domain email valid |
| Init SQL tidak jalan ulang | Volume pg-data sudah terinisialisasi | Gunakan migrasi atau reset via down -v |
| pgAdmin tidak terhubung | Host memakai localhost | Gunakan hostname postgres-db |
| Dump kosong / restore gagal | DB mati atau checksum tidak cocok | Cek status DB dan verifikasi sha256sum |

## Evaluasi dan Latihan Mandiri

**1. Mengapa init script tidak dijalankan ulang saat volume lama masih ada?**

Entrypoint image PostgreSQL hanya menjalankan initdb dan script di docker-entrypoint-initdb.d ketika direktori data (PGDATA) masih kosong. Bila klaster sudah terinisialisasi, entrypoint mendeteksi data yang ada dan melewati inisialisasi untuk mencegah penimpaan data. Akibatnya perubahan pada init script tidak berpengaruh pada volume lama; gunakan migrasi SQL atau recreates volume (down -v) bila data boleh dihapus.

**2. Apa risiko menaruh password database pada docker-compose.yml?**

Password akan tersimpan di version control, terlihat melalui docker compose config, docker inspect, environment proses, serta berpotensi muncul di log atau crash report. Siapa pun yang memiliki akses repository atau host dapat memperolehnya, dan rotasi menjadi sulit. Mitigasi: simpan pada .env yang dikecualikan dari git dengan permission ketat, atau gunakan Docker secrets/secret manager, dan rotasi credential secara berkala.

**3. Bagaimana cara membuktikan backup dapat dipulihkan?**

Lakukan restore nyata ke instance atau database terpisah, lalu verifikasi: checksum file cocok, pg_restore selesai tanpa error, jumlah baris dan isi data sesuai sumber. Pada praktikum, dump

direstore ke labdb_restore_test dan menghasilkan total_students = 3, sama dengan sumber. Backup baru dinyatakan valid setelah restore yang diuji, bukan hanya karena filenya ada.

**4. Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?**

pg_dump bersifat logis: menghasilkan snapshot skema dan data dalam format SQL atau custom, portabel lintas sistem dan toleran perbedaan versi, serta konsisten karena diambil dalam transaksi. Backup filesystem/volume menyalin direktori data secara fisik: lebih cepat untuk database besar, tetapi harus crash-consistent (database berhenti atau memakai pg_basebackup/PITR), terikat versi major, dan tidak portabel. Kesalahan menyalin saat database menulis dapat menghasilkan copy yang rusak.

**5. Apa dampak docker compose down -v terhadap database?**

Opsi -v menghapus named volume (pg-data dan pgadmin-data), sehingga seluruh data PostgreSQL hilang permanen. Ketika stack dinyalakan kembali, volume baru yang kosong dibuat dan init script dijalankan ulang. Karena bersifat destruktif, down -v hanya boleh dipakai bila data laboratorium memang tidak diperlukan; down tanpa -v tetap mempertahankan data.

# Penutup

## Kesimpulan

Praktikum Bab 5 berhasil menjalankan PostgreSQL dan pgAdmin pada Docker Compose dengan volume persisten, healthcheck, dan koneksi internal berbasis nama service. Init script membentuk tabel students beserta data awal, dan record baru (31230003) tetap ada setelah container dibuat ulang, membuktikan persistensi volume sekaligus menegaskan bahwa init script hanya berjalan pada volume kosong. Backup pg_dump custom-format beserta checksum SHA-256 tervalidasi, dan uji restore ke database terpisah berhasil dengan jumlah baris yang sesuai. Dari sisi keamanan, role read-only app_ro menolak operasi tulis, port hanya terikat ke loopback, dan credential terlindungi. Dua kendala nyata diselesaikan dengan diagnosis berbasis log: konflik port 5432 dan restart loop pgAdmin akibat domain email tak valid. Untuk production-like, disarankan memakai secret manager, migrasi terversi, backup off-host terenkripsi dengan uji restore berkala, serta pemantauan resource dan koneksi.

# Daftar Pustaka


Materi Ajar DevOps (DevSecOps Terintegrasi) — Bab 5. Politeknik Elektronika Negeri Surabaya, 2026.

PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation — pg_dump, pg_restore, pg_isready.

Docker Inc. (2026). Docker Documentation — Volumes dan Compose. https://docs.docker.com/

Akses terakhir: 9 Oktober 2026.